Let’s start with the public messages that allow you to open a bet, settle it, cancel it, and check its status. `Settle` matches the final result of the market with the bet. If the market result is in favor of the bet, the stake multiplied by the odds is added to the wallet:

```
object Bet {
  ...

  final case class Open(
      walletId: String,             ❶
      marketId: String,             ❷
      odds: Double,
      stake: Int,
      result: Int,                  ❸
      replyTo: ActorRef[Response])
      extends ReplyCommand
  final case class Settle(result: Int, replyTo: ActorRef[Response])
      extends ReplyCommand
  final case class Cancel(reason: String, replyTo: ActorRef[Response])
      extends ReplyCommand
  final case class GetState(replyTo: ActorRef[Response]) extends ReplyCommand
  ...
}
```

❶ The wallet to get funds from or add to

❷ The market to check for availability

❸ 0 represents winHome, 1 winAway, and 2 draw.

These messages can respond as follows

```
object Bet {
  ...

  sealed trait Response
  final case object Accepted extends Response
  final case class RequestUnaccepted(reason: String) extends Response
  final case class CurrentState(state: State) extends Response
  ...
}
```

depending on whether the bet is accepted or not when it is requested, or only the current state of the bet.